



The Evolution of the SSM State Matrix A

The general SSM form

A SSM (state space model) is basically a **Memory Machine** that:

- Keeps a hidden state h that acts like memory
- Updates that memory every step using a rule

A common (continuous-time) version SSM is typically written as:

$$\dot{h}(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t)$$

After discretisation with step size (Δ), we obtain the discrete-time recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t$$

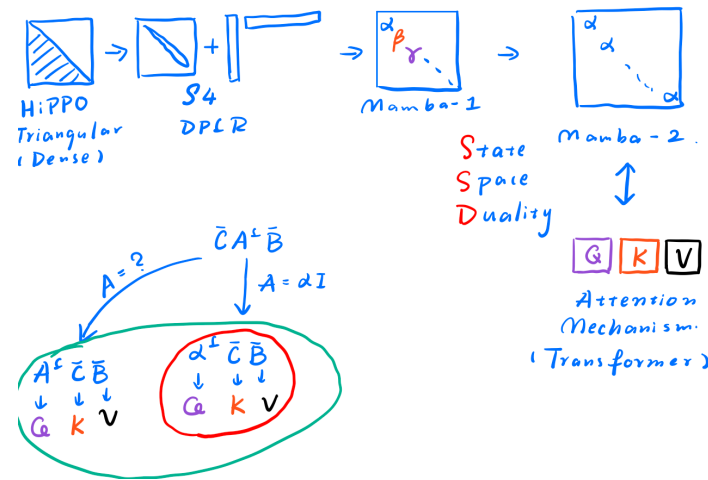
Intuitively, state matrix A defines the internal memory dynamics. It determines **how past information is retained, mixed, and forgotten** in the latent state h .

If A is stable (roughly: eigenvalues with negative real parts), the system's memory is well-behaved rather than exploding. Different structures for A (full, structured, diagonal, scalar) trade off **expressiveness vs efficiency**.

Therefore, if we want to track the evolution of SSMs, tracking the evolution of matrix A is a clean way to see what changed and why. In this tutorial, we are going to focus on the evolution of matrix A follow the line of works Hippo-LSSL-S4-Mamba01-Mamba02.

How A evolve

Stage	What A ($\bar{A}$) is	Input dependence?
HiPPO (LegS)	A : Lower-triangular (Dense)	No (LTI)
LSSL	A : HiPPO-LegS $\bar{A} = (I - \frac{\Delta}{2}A)^{-1}(I + \frac{\Delta}{2}A)$	No (LTI)
S4	DPLR parameterisation of A	No (LTI)
Mamba-01	A : Diagonal $\bar{A}_t = \exp(\Delta_t A)$	Effective ($\bar{A}_t$) depends on (Δ_t)
Mamba-02	$\bar{A}_t = a_t I$	Effective ($\bar{A}_t$) depends on (Δ_t)



We'll now walk through each stage, focusing solely on how A is chosen, parameterised, and used.

HiPPO: A as an optimal history compressor

HiPPO's key idea is:

*If you want to represent the entire history of a input signal $x(t)$ using a finite-dimensional state $h(t)$, you can do it by continuously projecting the input onto an **orthogonal polynomial basis**.*

A possible choice of basis is from the **Legendre (scaled) family (LegS)**, the resulting A is derived from properties of Legendre polynomials. Intuitively:

- The system is designed so that $h(t)$ stores a **uniformly informative** summary of the entire past.
- It is constructed to be a global memory compressor.

Mathematically, we could write down the representation of state matrix $A \in \mathbb{R}^{N \times N}$ (using zero-based indices $n, k \in \{0, \dots, N-1\}$):

$$A_{nk} = - \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

A nice consequence is that the diagonal simplifies to

$$A_{nn} = -(n+1),$$

so the diagonal is $-(1, 2, \dots, N)$.

Here we consider a simple case $N = 4$, expanding the formula gives

$$A = - \begin{bmatrix} 1 & 0 & 0 & 0 \\ \sqrt{3} & 2 & 0 & 0 \\ \sqrt{5} & \sqrt{15} & 3 & 0 \\ \sqrt{7} & \sqrt{21} & \sqrt{35} & 4 \end{bmatrix}.$$

We could easily observe that at this stage, A is a **dense lower-triangular matrix**.

If you are interested how A is derived step by step, please refer to [under construction].

LSSL: HiPPO A + Bilinear discretisation

What changes from HiPPO?

Sequence modelling needs a discrete-time system. LSSL uses HiPPO-inspired state matrix A as an initialisation / inductive bias and then discretises.

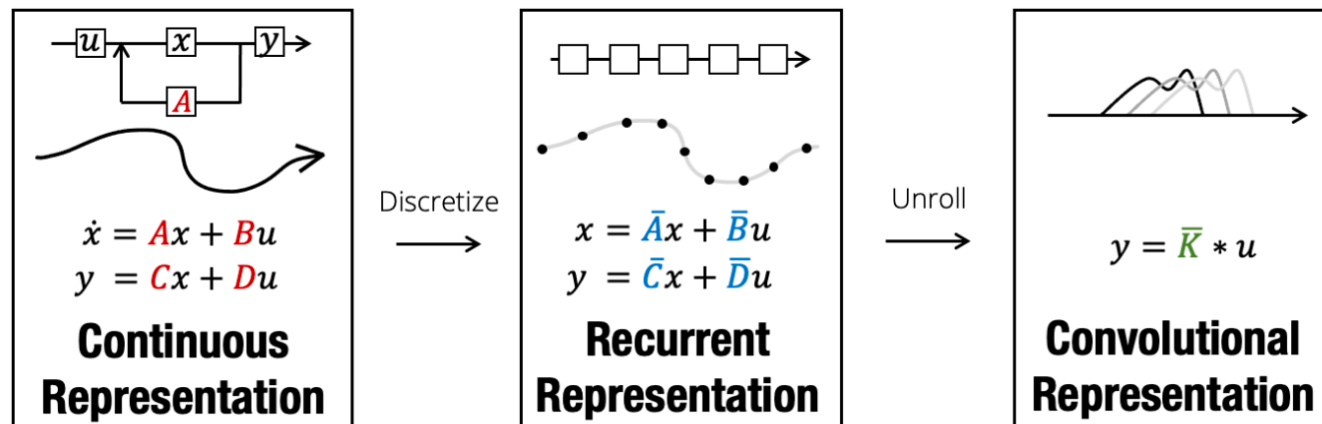
A common choice here is **Bilinear (Tustin) discretisation**, which produces:

$$\bar{A} = (I - \frac{\Delta}{2}A)^{-1}(I + \frac{\Delta}{2}A)$$

If you are interested in why we chose Bilinear method, please refer to:

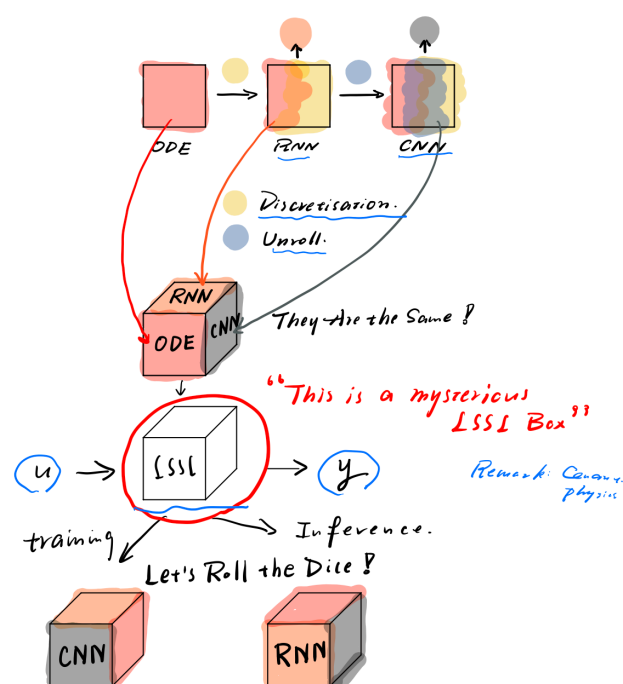
[appendix.pdf](#)

At this stage, A is still conceptually **time-invariant** (LTI).



For LTI systems, we could interpret the system from both RNN and CNN views. Thus at this stage SSMs can be computed either way, ideally:

- Inferencing like an RNN, and
- Training like a CNN.



S4: Making A computational friendly with DPLR

Up to HiPPO/LSSL, we had A that compresses the entire history really well — but it's **dense**, and doing anything like $(zI - A)^{-1}$ naively is expensive (people don't normally like matrix inversion).

In S4, we want to keep the HiPPO-LegS structure, while rewriting A so the expensive linear algebra becomes cheap. That rewrite is **DPLR** (*Diagonal Plus Low-Rank*).

Why do we suddenly care about $(zI - A)^{-1}$?

S4 often computes an **SSM convolution kernel** rather than scanning an RNN state.

The kernel here is:

$$K_t = C\bar{A}^t\bar{B}$$

Computing $\bar{A}^t$ directly is still expensive. Instead we jump to the z-transform:

$$\hat{K}(z) = \sum_{t \geq 0} K_t z^t = C(I - z\bar{A})^{-1}\bar{B}$$

So the whole problem reduces to repeatedly evaluating something like a **resolvent**:

$$(I - z\bar{A})^{-1}$$

Turn discrete resolvent into a continuous resolvent

S4 typically discretises in a way that lets us relate the discrete-time resolvent to a continuous-time one.

$$(I - z\bar{A})^{-1} \implies (\alpha(z)I - A)^{-1}$$

Why this helps: $\alpha(z)$ changes with z , but A **stays the same**.

So we need to evaluate $(\alpha(z)I - A)^{-1}$ for many z values efficiently. (we write $(zI - A)^{-1}$ for simplicity)

DPLR structure: rewrite A as diagonal + low-rank

$$A = \Lambda + PQ^\top \quad (\Lambda \text{ diagonal}, P, Q \in \mathbb{R}^{N \times r})$$

Think of this as:

- Λ : the easy part (diagonal $\rightarrow$ cheap inverses)
- $PQ^\top$: a small low-rank correction that recovers the HiPPO structure

With DPLR, we can use **Woodbury**.

Let

$$M = zI - A = (zI - \Lambda) - PQ^\top$$

Define $D = zI - \Lambda$ (still diagonal).

Woodbury says:

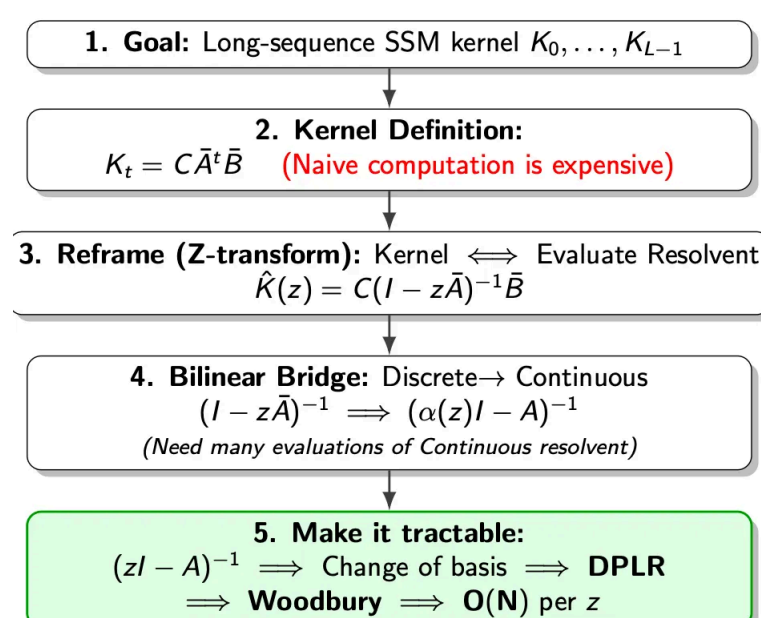
»

- D^{-1} is trivial (just invert diagonal entries).
- The inverse is now on an $(r \times r)$ matrix:

»

If $r = 1$, it's a scalar.

So per z , the cost drops from $O(N^3)$ to roughly $O(N)$.



If you are interested in step-by-step derivation of DPLR and Woodbury, please refer to:

[appendix.pdf](#)

From S4 to selective models: LTI becomes the bottleneck

At some point, the community realised a limitation: Pure LTI dynamics cannot ***select*** what to remember based on content.

From now on, we want to make Δ_t **input-dependent**, so the model can choose when to refresh memory and when to preserve it.

Mamba-01: Time dependent diagonal A + ZOH discretisation

The Mamba-01 paper describes two typical initialisations for the diagonal matrix A :

- **Real case:** $A_n = -(n + 1)$
- **Complex case:** $A_n = -\frac{1}{2} + ni$

An intuition is:

A is the system's built-in ***decay/oscillation rates***. It sets the time-scales of the latent state dimensions.

Let's consider a simple example here. Taking $N = 4$ and using the usual **zero-based index** $n = 0, 1, 2, 3$:

Real case: $A_n = -(n + 1)$

Complex case: $A_n = -\frac{1}{2} + ni$

$$A_{\text{real}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & -3 & 0 \\ 0 & 0 & 0 & -4 \end{bmatrix}$$

$$A_{\text{complex}} = \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & -\frac{1}{2} + i & 0 & 0 \\ 0 & 0 & -\frac{1}{2} + 2i & 0 \\ 0 & 0 & 0 & -\frac{1}{2} + 3i \end{bmatrix}$$

The paper's selective SSM makes Δ_t depend on the current token representation x_t , via

$$\Delta_t = \tau_{\Delta}(\text{bias} + s_{\Delta}(x_t)), \quad \tau_{\Delta} = \text{softplus}.$$

Δ_t is a per-token step size, so the model can decide when to update quickly and when to preserve memory.

- **If Δ_t is large:** the system takes a *big step* in time, which tends to **reset the current state** and **focus strongly on the current input**. Intuitively, the model is "selecting" this token and is more willing to forget what it was previously storing.
- **If Δ_t is small:** the system takes a *tiny step* in time, which tends to **preserve the current state** and **down-weight the current input**. Intuitively, the token is treated as more transient, and the model keeps its existing memory.

A helpful way to remember it is:

Large $\Delta_t \approx$ refresh memory from the current token. Small $\Delta_t \approx$ keep memory and let the token pass by.

Mamba-01 uses **ZOH discretisation**, where the discrete-time transition is

$$\bar{A}_t = \exp(\Delta_t A).$$

If you are interested in why we chose ZOH method, please refer to:

[appendix.pdf](#)

Thus, even if A is time-invariant, $\bar{A}_t$ becomes **input-dependent** as Δ_t is input-dependent.

Mamba-02: Scalar Identity A as SSD



So far so good, right? Mamba-01 is almost proudly **non-Transformer**. It just shows that a selective SSM can stand on its own as a serious sequence layer.

However, reality check. Unfortunately, the entire ecosystem is **Transformer-shaped**. GPUs love big matmuls; software stacks love blocks that look like attention/MLP; and reviewers/users often want things that plug into that world cleanly. And yes — there was a bit of drama. the original Mamba work (Mamba-01) was submitted to ICLR 2024, and the OpenReview thread shows a fairly intense review discussion.

Online discussion around that rejection also became a mini-meme, but let's not pretend social media is ground truth. If you want a quick break from the whole state matrix A thing, here are some clips of the discussion:

[D] Why is everybody surprised that Mamba got rejected from ICLR? Am I missing something?
178 votes, 38 comments. I'm not just trying to be contrarian either. I keep hearing this on Reddit, at work, on different online forums, etc. I also...

 https://www.reddit.com/r/MachineLearning/comments/1axsxeo/d_why_is_everybody_surprised_that_mamba_got/?utm_source=chatgpt.com



Welcome back.

Mamba-02 is thus basically:

If the world insists on thinking in attention blocks. fine, let's meet it halfway.

*Let's rewrite SSMs so they **look like attention**, and also can be computed using the same sort of hardware-friendly primitives.*

Let's recall the famous equation for causal attention:

$$Y = (L \circ QK^\top)V$$

What we here trying to answer is: *Can we choose an SSM parameterisation so that its sequence-to-sequence operator looks like masked attention?*

Recall: how a plain SSM becomes a causal matrix M

First we start from a discrete-time SSM:

$$h_t = A_t h_{t-1} + B_t x_t, \quad y_t = C_t^\top h_t$$

If we unroll this recurrence, y_j can be written as a sum of contributions from all earlier inputs x_i (for $(i \leq j)$):

$$y_j = \sum_{i \leq j} C_j^\top \underbrace{A_j A_{j-1} \cdots A_{i+1}}_{M_{ji}} B_i x_i$$

So the whole sequence map can be written as

$$y = Mx$$

where M is **lower-triangular** and its entries are

$$M_{ji} = C_j^\top A_j A_{j-1} \cdots A_{i+1} B_i \quad (j \geq i)$$

This step matters because it turns *SSM as a recurrence* into *SSM as a matrix operator*, which is exactly the attention uses.

At this point, M is a $(T \times T)$ matrix. If we treat it as an arbitrary dense matrix, computing Mx is $O(T^2)$ and storing M is also $O(T^2)$. That's not efficient for long sequences.

So we need structure. This is where **semiseparable** matrices come in.

A semiseparable matrix has low-rank structure in every causal block.

That low-rank structure lets you multiply by M faster than $(O(T^2))$, without explicitly forming M .

So far we have a **general expression** for M_{ji} . Now we look for a specific parameterisation of A_t that makes M factorise into attention-ish expression.

The paper points out a particularly clean special case:

$$A_t = a_t I \quad (\text{scalar} \times \text{identity})$$

Then the product collapses:

$$A_j A_{j-1} \cdots A_{i+1} = (a_j a_{j-1} \cdots a_{i+1}) I = a_{j:i+1} I$$

and therefore

$$M_{ji} = a_{j:i+1} (C_j^\top B_i)$$

This is the Attention-shaped factorisation we are looking for:

- $(C_j^\top B_i)$ is a **content score** (a dot product: query–key style),
- $(a_{j:i+1})$ is a purely causal **mask/decay term** (depends only on positions between i and j).

Equivalently, you can think of

$$M \approx L_a \circ (CB^\top)$$

where L_a is a structured causal matrix built from the scalar chain $(a_{j:i+1})$, and $(\circ)$ is elementwise multiplication.

Causal attention is often written as:

$$Y = (L \circ QK^\top)V$$

so the mapping is immediate:

- $(Q \leftrightarrow C)$
- $(K \leftrightarrow B)$
- $(V \leftrightarrow X)$
- $(L \leftrightarrow L_a)$

Remark. $A_t = a_t I$ is a special case, not the only possible route.

Choosing $(A_t = a_t I)$ is a particularly neat sufficient condition that makes the SSM operator look like masked attention.

It demonstrates **Existence**: there exists a structured SSM (within the semiseparable family) whose M is attention-shaped.

It does **not** claim uniqueness. There may be other parameterisations of A_t (or other structured matrix families) that also yield attention-like decompositions, or different fast algorithms. The paper focuses on scalar-identity as it gives a clean semiseparable form and strong efficiency properties.

Further reading:

If you'd like to trace the exact lineage of the A story (HiPPO $\rightarrow$ LSSL $\rightarrow$ S4 $\rightarrow$ Mamba), these are the original papers to look up:

- HiPPO: <https://arxiv.org/abs/2008.07669>
- LSSL: <https://arxiv.org/abs/2111.00396>
(This is the paper that popularised the “structured state space” pipeline and the LSSL formulation.)
- S4: <https://arxiv.org/abs/2111.00396>
(Key idea: the DPLR / structured parameterisation of (A) to make kernel/state updates efficient.)
- Mamba-01: <https://arxiv.org/abs/2312.00752>
- Mamba-02: <https://arxiv.org/abs/2405.21060>

